

```
#!/usr/bin/env python3
"""
app.py – Web app wrapper around autoclip.py

Run locally:
    uvicorn app:app --host 0.0.0.0 --port 8000

Then open http://localhost:8000 in a browser.

Jobs run in a background thread per request. State is
kept in memory,
which is fine for personal/single-user use. If you
need multiple
concurrent users or restarts without losing job
history, swap JOBS
for a small SQLite table later.
"""

import threading
import uuid
from pathlib import Path
from typing import Optional

from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field

import autoclip

BASE_DIR = Path(__file__).parent
OUTPUT_ROOT = BASE_DIR / "job_output"
OUTPUT_ROOT.mkdir(exist_ok=True)

app = FastAPI(title="autoclip")

# in-memory job store: job_id -> dict
JOBS: dict[str, dict] = {}
```

```

JOBS_LOCK = threading.Lock()

class NewJobRequest(BaseModel):
    url: str
    num_clips: int = Field(default=5, ge=1, le=20)
    min_sec: float = Field(default=20, gt=0)
    max_sec: float = Field(default=60, gt=0)
    whisper_model: str = Field(default="base")

def _set_job(job_id: str, **fields):
    with JOBS_LOCK:
        JOBS[job_id].update(fields)

def _run_job(job_id: str, req: NewJobRequest):
    out_dir = OUTPUT_ROOT / job_id
    workdir = out_dir / "work"

    def progress_cb(status: str, detail: str):
        _set_job(job_id, status=status, detail=detail)

    try:
        manifest = autoclip.run(
            url=req.url,
            num_clips=req.num_clips,
            min_sec=req.min_sec,
            max_sec=req.max_sec,
            out_dir=str(out_dir),
            whisper_model=req.whisper_model,
            keep_source=False,
            progress_cb=progress_cb,
            workdir=workdir,
        )
        _set_job(job_id, status="done", detail="",
            clips=manifest or [])
    except Exception as e:
        _set_job(job_id, status="error",
            detail=str(e))

```

```

@app.post("/api/jobs")
def create_job(req: NewJobRequest):
    if req.min_sec > req.max_sec:
        raise HTTPException(400, "min_sec cannot be
greater than max_sec")

    job_id = uuid.uuid4().hex[:12]
    with JOBS_LOCK:
        JOBS[job_id] = {
            "id": job_id,
            "status": "queued",
            "detail": "",
            "clips": [],
            "request": req.model_dump(),
        }

    thread = threading.Thread(target=_run_job, args=
(job_id, req), daemon=True)
    thread.start()
    return {"job_id": job_id}

@app.get("/api/jobs/{job_id}")
def get_job(job_id: str):
    with JOBS_LOCK:
        job = JOBS.get(job_id)
    if not job:
        raise HTTPException(404, "job not found")

    clips = [
        {**c, "url":
f"/api/jobs/{job_id}/clips/{c['file']}"
        for c in job.get("clips", [])
    ]
    return {
        "id": job["id"],
        "status": job["status"],
        "detail": job["detail"],
        "clips": clips,
    }

```

```
@app.get("/api/jobs/{job_id}/clips/{filename}")
def get_clip_file(job_id: str, filename: str):
    path = OUTPUT_ROOT / job_id / filename
    if not path.is_file():
        raise HTTPException(404, "clip not found")
    return FileResponse(path, media_type="video/mp4",
filename=filename)

# Serve the frontend last so it doesn't shadow the
# /api routes above
app.mount("/", StaticFiles(directory=str(BASE_DIR /
"static"), html=True), name="static")
```